U
A

X
E
R
CI
C
E
S
/
Fi
n
al
Pr
oj
ec
t.
P
ar
t
1

Visión por Computador (2026-27, Graduado/a en Ingeniería del Software. Plan 2023 Todos los grupos y Graduado/a en Ingeniería Informática. Plan 2023 Todos los grupos)

Final Project. Part 1

Part 1: fruit detection in images

This page describes the first part of the final project, *Detecting and counting fruits on trees*, so read the introduction to the project first if you have not done it yet.

Before counting the fruits of a whole tree, we need to solve a more basic problem: finding them in a single image. This is the goal of Part 1. Your task is to build a method that receives an image of a fruit tree and, for every visible fruit, returns a **mask** with the pixels of the fruit, its **species** and a **confidence score** (see Fig. 1). In computer vision, this task is known as **instance segmentation**: it does not only tell which pixels belong to fruits, but also which pixels belong to each individual fruit. The images can contain fruits of four species:

`category_id`	**Species**
1	orange
2	apple
3	pear
4	lemon



Fig. 1. Part 1 at a glance. Top: images of the train set. Bottom: the expected output of your method, with the mask, species and confidence score of every visible fruit (the scores shown are only illustrative).

As explained in the introduction, you will work with two sets of data. The **train** set is the one you receive, to develop, train and validate your method on your own (Section 2). The **test** set contains images of other trees and is never made public: we use it to evaluate the methods of all groups, by running your code on our machine (Section 5.2).

The rest of this page follows the path of your work, step by step (see Fig. 2). First, we explain how to get the data and the starter kit (Section 1) and what the data look like (Section 2). Then, we describe how to develop your detector: what it must produce and how to organize its code (Section 3). Next, we explain how to

evaluate it, which you will do many times while you improve it (Section 4). Finally, we explain how to submit your work (Section 5) and the rules of the challenge (Section 6).

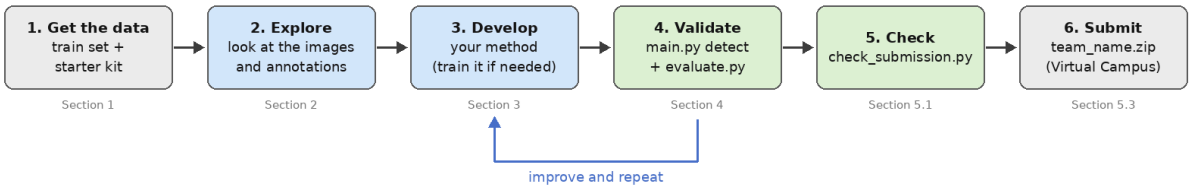


Fig. 2. The steps of your work in Part 1, with the section of this page that explains each of them. Steps 3 and 4 form a loop: you improve your method and measure it again on your validation images until you are happy with it.

1. Getting started

To start working, you need the train set and a few tools that we provide in a starter kit. Getting them takes three steps:

1. Download `fruits_part1.zip` (about 70 MB) from the Virtual Campus and unzip it.
2. Get the starter kit from the course repository, in the folder `Final project/Part 1/` of `uma_cv_2027` (`git pull` if you already have a clone).
3. Install the kit's requirements in a Python 3 environment. You can use any recent version of Python 3, but keep in mind that we evaluate your submission with **Python 3.12** (Section 5.2), so using that version too avoids surprises. On Linux and macOS:

   ```
   python3.12 -m venv .venv
   source .venv/bin/activate
   pip install -r "Final project/Part 1/requirements.txt"
   ```

   On Windows:

   ```
   py -3.12 -m venv .venv
   .venv\Scripts\activate
   pip install -r "Final project/Part 1/requirements.txt"
   ```

The starter kit contains the same tools that we use to check and score your work, so you can measure your progress exactly as we will:

File	Use
`evaluate.py`	Scores a predictions file against the annotations, with the official metrics (Section 4)
`check_submission.py`	Checks that your submission follows the rules, before you submit it (Section 5.1)
`visualize.py`	Draws annotations or predictions over the images
`example_submission/`	A minimal submission that follows the rules. Its detector is deliberately naive: it shows the format, not a method

2. The data

Good methods start with a good understanding of the data. In this section we describe what the train set contains, the format of its annotations and the criteria followed to annotate the fruits.

Once unzipped, the train set of Part 1 has the following structure:

```
fruits_part1/
  train/
    images/            img_000001.jpg ... (200 images)
    images.json        list of images and categories, without annotations
    annotations.json   COCO annotations
```

The `images` folder contains 200 JPEG images of 1280×960 pixels. They were taken from all sides of the trees and at different distances, from views of the whole tree at several meters to close-ups about one meter away from the fruits. The lighting, the time of day and the background also change from image to image. File names are random, so they do not reveal anything about the content of the image. The test set has exactly the same format, with images of other trees, but its annotations are never made public.

2.1 Annotation format

Both JSON files of the train set, `images.json` and `annotations.json`, follow the format of COCO (*Common Objects in Context*). COCO started as a large dataset of everyday images published by Microsoft in 2014, but its annotation format became a de facto standard: today most detection and segmentation datasets, libraries and models read and write it. Learning it is therefore useful well beyond this project.

2.1.1 images.json

The simpler of the two files is `images.json`. It only says which images there are and which categories they can contain:

```
{
  "categories": [
    {"id": 1, "name": "orange"},
    {"id": 2, "name": "apple"},
    {"id": 3, "name": "pear"},
    {"id": 4, "name": "lemon"}
  ],
  "images": [
    {"id": 1, "file_name": "img_000001.jpg", "width": 1280, "height": 960},
    ...
    {"id": 200, "file_name": "img_000200.jpg", "width": 1280, "height": 960}
  ]
}
```

The `categories` list contains the four species, each with its `id` and its `name`. The `images` list describes every image of the set: its `id`, the name of its file in the `images` folder, and its `width` and `height` in pixels. The `id` of an image is the number that identifies it everywhere else: in the annotations and, more importantly, in the predictions of your method, which refer to images by their `id` and not by their file name (Section 3.1).

In the train set, `images.json` may look redundant, since `annotations.json` contains the same two lists. However, it is the file your program receives at evaluation time, together with the test images (Section 3.2). Having it in the train set too lets you run your program on the train set exactly as we will run it on the test set.

2.1.2 annotations.json

The file `annotations.json` contains the same two lists as `images.json`, plus a third one with the annotations. The three lists are linked to each other through their numeric identifiers:

```
{
  "categories": [...],
  "images": [...],
  "annotations": [{
    "id": 1, "image_id": 1, "category_id": 2,
    "segmentation": {"size": [960, 1280], "counts": "..."},
    "area": 812, "bbox": [x, y, w, h], "iscrowd": 0,
    "fruit_id": 37
  }, ...]
}
```

The `annotations` list contains one entry per fruit and image, that is, per **instance**: a fruit that appears in three images has three annotations. Each annotation points to its image through `image_id` and to its species through `category_id`. So, to get all the fruits of the image `img_000001.jpg`, you look up its `id` in `images` and then take all the annotations whose `image_id` has that value (the `COCO` class of `pycocotools` does it for you, as shown below).

These are the fields of each annotation:

Field	Meaning
`id`	Identifier of the annotation, unique in the file
`image_id`	The image where the fruit appears (an `id` of `images`)
`category_id`	The species of the fruit (an `id` of `categories`)
`segmentation`	The mask of the visible pixels of the fruit, encoded in RLE (see below)
`area`	Number of pixels of the mask
`bbox`	Bounding box of the mask, as `[x, y, width, height]` in pixels, where `(x, y)` is its top-left corner and the origin of the image is its top-left pixel
`iscrowd`	1 for the tiny fragments that the evaluation ignores (Section 2.2), 0 for the rest. In the original COCO it marks groups of objects annotated together, which the evaluation also ignores
`fruit_id`	Identifier of the physical fruit: the same fruit has the same `fruit_id` in every image where it appears. It is not part of standard COCO, and you do not need it for Part 1, but it will be useful in Part 2

The only field that needs some more explanation is `segmentation`. COCO can store a mask in two ways: as a list of polygons that follow its outline, or as a **run-length encoding** (RLE). We use RLE because it represents any mask exactly, including fruits split into several pieces by leaves. The idea is simple. The mask is read pixel by pixel, column by column (from top to bottom, and then from left to right), and instead of storing every pixel, RLE stores the lengths of the runs of equal values, always starting with a run of zeros. For example, this mask of 3 rows and 4 columns:

```
0 1 1 0
0 1 1 0
0 0 0 0
```

is read, column by column, as `0 0 0 | 1 1 0 | 1 1 0 | 0 0 0`, that is, 3 zeros, 2 ones, 1 zero, 2 ones and 4 zeros, so its RLE is `{"size": [3, 4], "counts": [3, 2, 1, 2, 4]}` (see Fig. 3). For a 1280×960 image this is far more compact than storing the 1,228,800 pixels of the mask. In our files, the list of `counts` is further compressed into a short string, which is why you see `"counts": "..."` with letters and symbols. You never need to read or write these strings by hand: `pycocotools` converts between masks and RLE in both directions, as shown below and in Section 3.1.

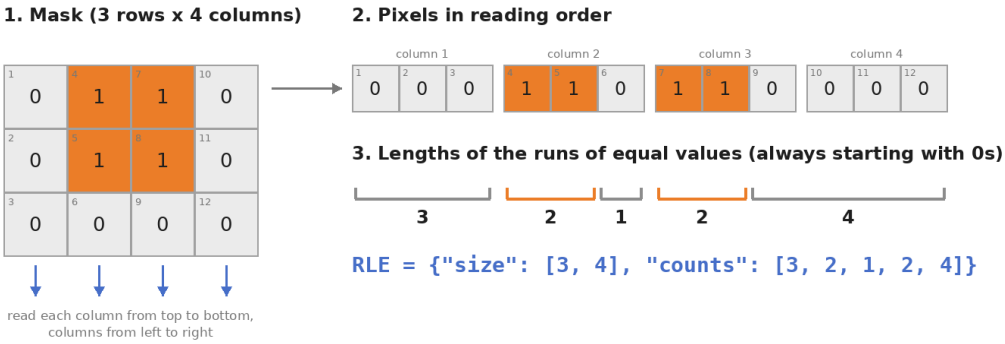


Fig. 3. Run-length encoding (RLE) of a mask of 3 rows and 4 columns. The pixels are read column by column, from top to bottom, and the RLE stores the lengths of the runs of equal values, starting with a run of zeros.

In practice, the easiest way to work with the annotations is the `COCO` class of `pycocotools`, which loads the file and gives access to the images and to the mask of every fruit. For example:

```
from pycocotools.coco import COCO

coco = COCO("fruits_part1/train/annotations.json")
img_id = coco.getImgIds()[0]
info = coco.loadImgs(img_id)[0]                      # file_name, width, height
anns = coco.loadAnns(coco.getAnnIds(imgIds=img_id, iscrowd=False))
mask = coco.annToMask(anns[0])                       # (960, 1280) array of 0/1
print(info["file_name"], len(anns), "fruits,", coco.cats[anns[0]["category_id"]]["name"])
```

It is also a good idea to look at the annotations before designing your method. The starter kit draws them over the images:

```
python visualize.py --images fruits_part1/train/images --coco fruits_part1/train/annotations.json --out vis/
```

It writes one image per input image in the `vis` folder, with every fruit in its own color and labeled with its species. Drawing all the images can take a while, so you can add `--limit 10` to draw only the first 10, or `--no-labels` to hide the labels, which can clutter images with many small fruits.

2.2 What counts as a fruit

Annotating fruits on a tree is less obvious than it seems, since many of them are only partly visible. These are the criteria followed in the annotations:

1. **Every visible fruit is annotated** with one mask, including fruits cut by the image border.
2. **Masks cover only the visible pixels.** A fruit split by a leaf has a single mask made of several pieces. Leaves, branches and the stalk that joins the fruit to the branch are never part of the mask.
3. **Tiny fragments are ignored.** A fruit with fewer than 20 visible pixels is annotated with `iscrowd: 1`. In the evaluation, these regions are "don't care": detecting them gives no credit and missing them is not penalized. Many fruits deep in the canopy fall in this group.

As you will see, images can contain more than 100 visible fruits, and many of them are small and partly hidden by leaves (see Fig. 4). Dealing with these small, occluded fruits is one of the main challenges of this part.



Fig. 4. A training image and its annotations. Each fruit has its own color; the yellow box marks the zoomed area (bottom). Fruits partly hidden by leaves are annotated only where they are visible.

3. Developing your detector

Now that you know the data, it is time to build your detector. This section explains what it must produce, how we will start it and how to organize its code. Organize your project like this from the very beginning: the folder you develop in is the same folder you will submit (Section 5).

3.1 What your detector must produce

For every image, your method returns a list of **detections**, one per fruit it finds. Each detection has three elements:

- a **mask** with the pixels of the fruit,
- a **category**, one of the four species,
- a **score** in [0, 1]: your confidence that the detection is a fruit of that category. It ranks your detections in the evaluation, so it matters.

Some methods, for example many classical computer vision pipelines, do not produce a confidence for their detections. In that case, simply set the score of every detection to 1.

All the detections of all the images are written to a single JSON file, a list in the COCO results format:

```
[
  {"image_id": 1, "category_id": 2, "score": 0.93,
   "segmentation": {"size": [960, 1280], "counts": "..."}},
  ...
]
```

For the file to be valid, there can be **at most 300 detections per image** (above that number, we drop the lowest-scoring ones), the **image and category ids** must be the ones in `images.json`, and the `size` of each mask must be the `[height, width]` **of its image**.

Masks are written in the same RLE format as the annotations. This is how to encode a boolean mask of shape `(height, width)`:

```
import numpy as np
from pycocotools import mask as mask_utils

rle = mask_utils.encode(np.asfortranarray(mask.astype(np.uint8)))
rle["counts"] = rle["counts"].decode("ascii")       # bytes -> str, so it can go into JSON
detection = {"image_id": img_id, "category_id": 2, "score": 0.93, "segmentation": rle}
```

3.2 The `main.py` program

Your detector is a Python program, `main.py`, that we start automatically for every group, so all of them must be started in the same way. We run your method with exactly this command, from inside your folder:

```
python main.py detect --input <input folder> --output <predictions file> --device cuda
```

- `--input` is a folder with the same layout as `fruits_part1/train/`, but without `annotations.json`:
  ```
  <input folder>/
    images/        img_*.jpg
    images.json
  ```
- `--output` is the path of the predictions file to write (Section 3.1).
- `--device` is `cuda` or `cpu`. On our machine it is always `cuda`.
- The program must end with exit code 0.

The following minimal `main.py` shows the structure of the program. It reads the list of images, calls a `detect` function for each of them and writes the predictions file. You only have to fill in the detector:

```
import argparse
import json
from pathlib import Path


def detect(image_path):
    """Detects the fruits of one image.

    Returns a list with one tuple (mask, category_id, score) per detected fruit:
        mask         numpy array of booleans, shape (height, width): True on the pixels of the fruit
        category_id  int: 1 = orange, 2 = apple, 3 = pear, 4 = lemon
        score        float in [0, 1]: your confidence in the detection (1 if your method has none)

    For example, for an image with two apples and one pear:
        [(mask_1, 2, 0.95), (mask_2, 2, 0.81), (mask_3, 3, 0.64)]
    """
    detections = []
    ...                                     # your detector goes here
    return detections


def main():
    p = argparse.ArgumentParser()
    sub = p.add_subparsers(dest="task", required=True)
    d = sub.add_parser("detect")
    d.add_argument("--input", type=Path, required=True)
    d.add_argument("--output", type=Path, required=True)
    d.add_argument("--device", default="cuda", choices=["cuda", "cpu"])
    args = p.parse_args()

    images = json.loads((args.input / "images.json").read_text())["images"]
    results = []
    for im in images:
        for mask, category_id, score in detect(args.input / "images" / im["file_name"]):
            results.append(...)             # see the encoding example in Section 3.1
    args.output.write_text(json.dumps(results))


if __name__ == "__main__":
    main()
```

In Part 2, we will add a second command to this same program, `python main.py count ...`, so your project will keep growing in the same folder.

3.3 Project folder

Your project lives in a single folder, `team_<name>/`, with this structure:

```
team_<name>/
  README.md          members, and what your project does (Section 3.4)
  requirements.txt   Python dependencies, with exact versions (==)
  main.py            entry point (Section 3.2)
  src/               (optional) the rest of your code, organized as you like
  weights/           (optional) trained models and any pretrained weights you use
```

Keep in mind the following requirements:

- **Everything is written in Python.** There is no installation step other than `requirements.txt`.
- **It must work with Python 3.12**, the version we use to run it, even if you developed it with another one. Pinned versions in `requirements.txt` do not exist for every Python version (for example, old releases of `numpy` or `torch` cannot be installed on Python 3.12), so check it with `check_submission.py` (Section 5.1).
- **If your method is trained, include the code you used to train it.** We only run the detection, which loads your trained weights, but we need the training code to see how you obtained those weights. Explain in the README how to run it (Section 3.4). If your method does not need training, for example because it is based on fixed rules or thresholds, you can skip this point.
- **Size limit:** 500 MB for the whole folder.
- **Not allowed in the zip:** the dataset, virtual environments (`.venv/`), caches (`__pycache__/`).

3.4 README

There is no written report. Instead, the `README.md` of your project briefly explains it. It must include:

- the members of the group;
- how your method works, and why you chose it;
- if your method is trained: how you trained it and with which data, and how to run your training code (the command to start it, its main options and where it saves the weights), so that we could reproduce it;
- the scores you obtained on your validation images, or on the whole train set if your method does not learn from the data (Section 4.1);
- every pretrained model, external dataset or library beyond the usual ones that you use, with a link.

4. Evaluating your detector

Once your detector produces predictions, the next question is how good they are. You will ask yourself this question many times, every time you change something in your method (the loop of Fig. 2). This section explains how to score your predictions and what the metrics measure.

4.1 Scoring your predictions

First, run your detector on the train set, exactly as we will run it on the test set. Then, the script `evaluate.py` of the starter kit compares the predictions file with the annotations and computes the official metrics:

```
python main.py detect --input fruits_part1/train --output predictions.json --device cuda
python evaluate.py --gt fruits_part1/train/annotations.json --pred predictions.json
```

If your method learns anything from the data, for example a network that you train or thresholds that you tune automatically, keep part of the train set for **validation**. Use those images only to measure your method, never to train it: a method scored on the same images it learned from looks better than it really is. With `--images <list of ids>` (or `--images-file`, with one id per line), `evaluate.py` scores only those images. If your method does not learn from the data, for example a classical pipeline with rules and thresholds that you set by hand, you do not need this split and can score it on the whole train set.

In both cases, bear in mind that these images come from the training trees, so your score on the test set will usually be lower.

4.2 Metrics

The metrics are those of COCO instance segmentation, the standard way of evaluating this task, computed with `pycocotools` on the masks. They may look complicated at first, but they are built step by step from three simple questions: when is a detection correct, how good is a list of detections ordered by score, and how do we summarize everything in a single number.

4.2.1 When is a detection correct?

To decide whether a detection has found a fruit, we compare its mask with the mask of the annotated fruit using the **intersection over union** (IoU): the number of pixels that belong to both masks, divided by the number of pixels that belong to at least one of them.

$$\text{IoU} = |\text{predicted mask} \cap \text{annotated mask}| \,/\, |\text{predicted mask} \cup \text{annotated mask}|$$

The IoU is 1 when both masks are identical and 0 when they do not overlap at all (see Fig. 5).

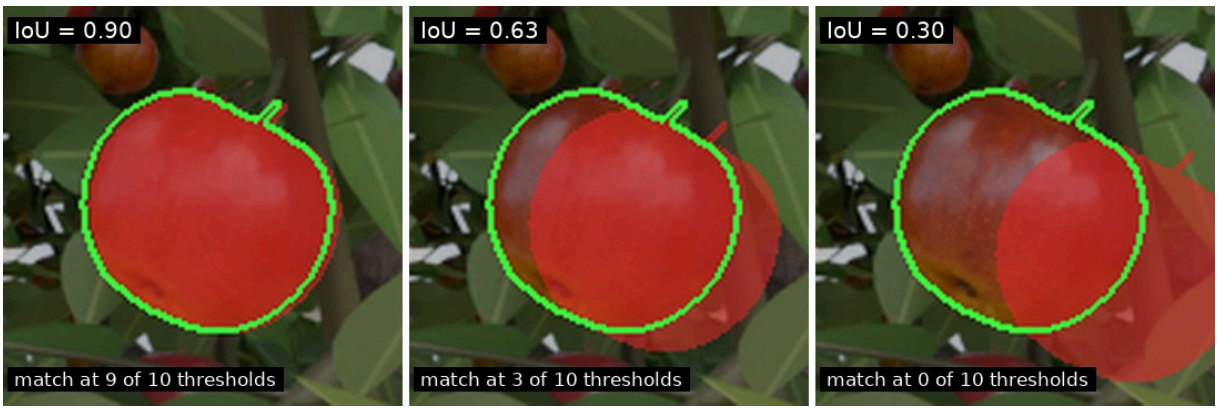


Fig. 5. Mask IoU between an annotated fruit (green outline) and three predicted masks (red). The more the masks overlap, the higher the IoU, and the more IoU thresholds the detection passes.

A detection is a **true positive** (TP) when it has the **same species** as an annotated fruit and their IoU reaches a **threshold**, for example 0.5. Detections are processed from the highest score to the lowest, and each one is matched with the annotated fruit it overlaps most among those not matched yet, so a fruit can only be found once. A detection that matches no fruit is a **false positive** (FP): a detection on the leaves, a second detection of a fruit already found, or a fruit with the wrong species. An annotated fruit that no detection matches is a **false negative** (FN), that is, a missed fruit. Detections on the tiny ignored fragments (Section 2.2) are not counted at all. Fig. 6 shows all these cases. Note that, unlike in classification, there are no **true negatives** (TN) in detection: the background is not made of a countable set of objects, so the metrics only use TP, FP and FN.

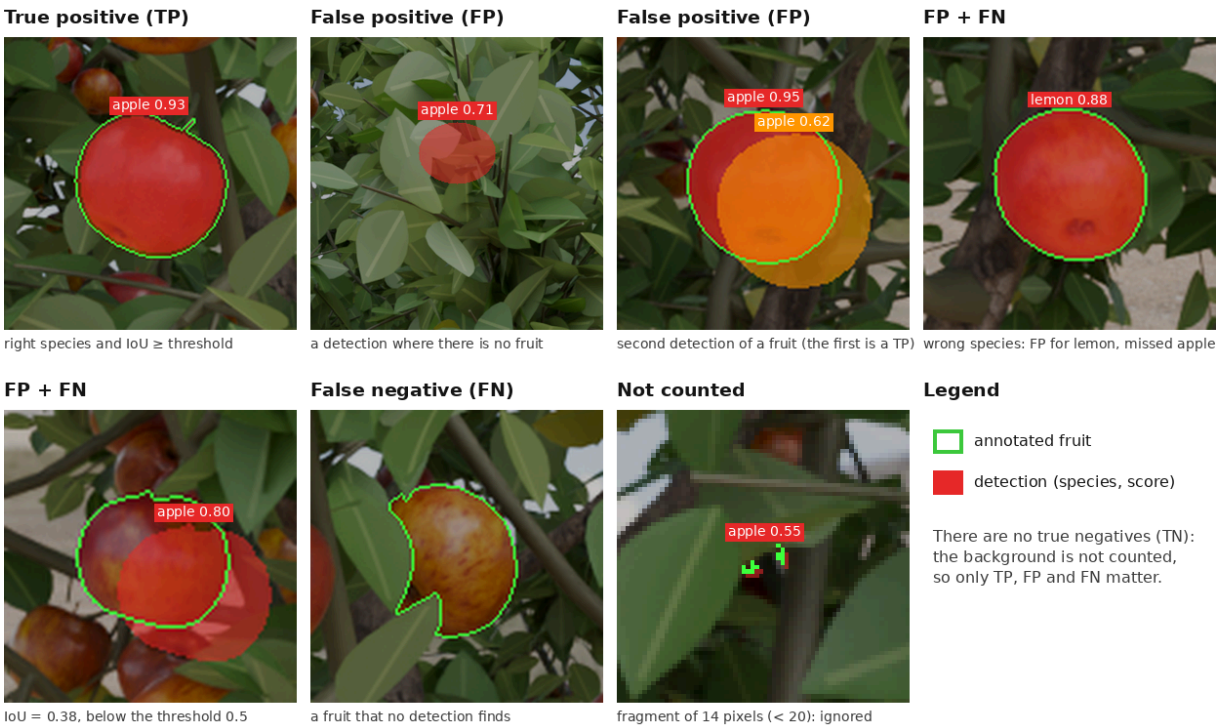


Fig. 6. The possible outcomes of a detection, at an IoU threshold of 0.5. A detection with the wrong species or with an imprecise mask is a false positive, and the fruit it should have found counts as missed (a false negative).

4.2.2 Precision, recall and average precision

With these definitions we can compute two classical measures. The **precision** is the fraction of detections that are correct, TP / (TP + FP), and the **recall** is the fraction of fruits that are found, TP / (TP + FN). They pull in opposite directions: a method that returns only its most confident detections has a high precision but a low recall, and one that returns everything it sees has a high recall but a low precision.

This is where the **score** of your detections comes in. The detections are sorted by score, and precision and recall are computed after each of them, as if you only kept the detections down to that one. For example, suppose that the images contain 4 apples and your method returns 5 apple detections:

Rank	Score	Correct?	Precision	Recall
1	0.95	yes (TP)	1/1 = 1.00	1/4 = 0.25
2	0.90	yes (TP)	2/2 = 1.00	2/4 = 0.50
3	0.80	no (FP)	2/3 = 0.67	2/4 = 0.50
4	0.60	yes (TP)	3/4 = 0.75	3/4 = 0.75
5	0.40	no (FP)	3/5 = 0.60	3/4 = 0.75

These pairs of values form the **precision-recall curve**, and the **average precision** (AP) is, roughly, the area under it: the average precision over all recall values from 0 to 1. In the example, the precision is 1.00 up to a recall of 0.50, 0.75 up to a recall of 0.75, and 0 beyond that, since one apple is never found. This gives AP ≈ 0.69 (COCO averages the precision at 101 recall values, 0, 0.01, ..., 1). An AP of 1 means that all the fruits are found and that every correct detection has a higher score than every wrong one. Therefore, AP rewards finding as many fruits as possible **and** giving higher scores to the detections you are more sure of.

4.2.3 The main metric

The AP above depends on two choices: the species and the IoU threshold. The **main metric** of the challenge, **mask AP@[.50:.95]** (`AP` in the output of `evaluate.py`), removes both choices by averaging. It computes the AP of each of the four species at each of the ten IoU thresholds 0.50, 0.55, ..., 0.95, and averages these 40 values. Averaging over thresholds means that a detection whose mask is only roughly right counts as correct at the low thresholds but not at the high ones, so **precise masks score higher**. Averaging over species means that all four species weigh the same, however many fruits each one has. And since a detection with the wrong species is a false positive for its species and leaves its fruit missed in the right one, the metric rewards good masks and correct species together. Up to 300 detections per image are taken into account.

4.2.4 Other metrics

A single number rarely tells the whole story, so `evaluate.py` also prints the following metrics, with the same names as in its output. They help you understand where your method fails, for example whether it misses small fruits, draws imprecise masks or confuses species:

Name	Meaning
`AP`	The main metric, mask AP@[.50:.95] (Section 4.2.3)
`AP50`, `AP75`	AP with a single IoU threshold, 0.5 or 0.75. `AP50` accepts any reasonable overlap, while `AP75` needs precise masks. A large gap between them means that your method finds the fruits but outlines them poorly
`AP_orange`, `AP_apple`, `AP_pear`, `AP_lemon`	AP of each species: which fruits your method handles worse

Name	Meaning
`AP_agnostic`	AP when the four species are merged into a single class, "fruit". It measures detection and segmentation only. If it is much higher than `AP`, your method finds the fruits but gets their species wrong
`accuracy`	Among the detections that overlap a fruit with IoU ≥ 0.5, the fraction with the right species
`AP_small`, `AP_medium`, `AP_large`	AP of the fruits whose visible area is below 32×32 pixels, between 32×32 and 96×96 pixels, or above 96×96 pixels. It shows how your method copes with distant or hidden fruits. A value of `nan` means that there are no fruits of that size in the evaluated images
`images`, `detections`, `fruits`	Number of evaluated images, of your detections in them, and of annotated fruits (without the ignored fragments)

5. Submitting your work

We do not ask you for your predictions on the test set, since you will never see it. Instead, you submit your code, and we run it ourselves. When you are happy with your detector, three steps remain: check that your submission works, understand how we will run it, and upload it.

5.1 Checking your submission

To avoid surprises, the starter kit includes a script that reproduces our process on your machine:

```
python check_submission.py team_<name>/ --data fruits_part1/train
```

It goes through the same steps that we follow (Section 5.2):

1. checks the folder structure and size;
2. installs `requirements.txt` in a new virtual environment with Python 3.12 (it warns you if Python 3.12 is not installed on your machine and uses your version instead);
3. runs `main.py detect` on 10 images of the train set and measures the time;
4. validates the predictions file and prints its score.

The 10 images are chosen at random from the train set given with `--data`, and the score is computed against their annotations, so it is only a quick sanity check, not an estimate of your test score. If you do not have a GPU, add `--device cpu`. The script works on Linux, macOS and Windows, but it does not disable the internet, so make sure yourself that your program does not download anything while it runs: on our machine, it will not have internet access (Section 5.2).

5.2 How we run your submission

To evaluate all groups under the same conditions, and to keep the test set hidden, your code runs on our machine in two steps (see Fig. 7):

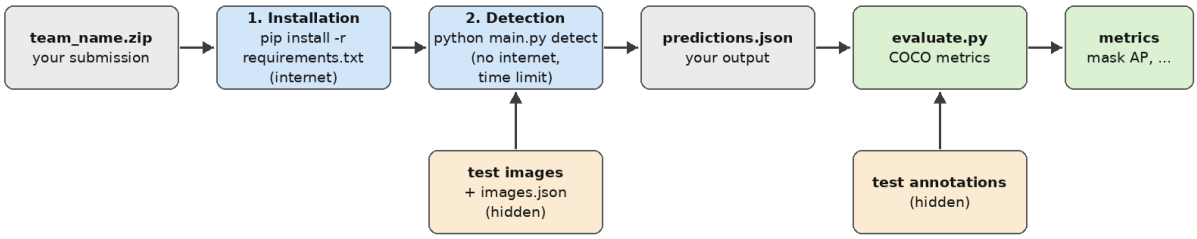


Fig. 7. How we run and score your submission. Internet access is only available during the installation, and the test annotations never leave our machine.

1. **Installation.** In a new virtual environment with Python 3.12, on a machine with Ubuntu 24.04 and an NVIDIA GPU, we run:
   ```
   pip install -r requirements.txt
   ```
   Internet access is available only in this step. If you need packages from another index (for example, PyTorch builds for a specific CUDA version), add the line `--extra-index-url <url>` to `requirements.txt`.

2. **Detection.** We run the command of Section 3.2 on the images of the test set, **without internet access**, with these limits:

Resource	Limit
GPU	NVIDIA RTX 2060, 6 GB of memory
CPU and memory	4 cores, 12 GB of RAM
Time	20 minutes for the whole test set, including loading your model
Files	your program must not modify the input folder; it can write only the output file and temporary files in `/tmp`

   So your code cannot download anything at this point. This is a common pitfall: many libraries download pretrained weights the first time a model is created, so save those files in `weights/` and load them from there.

If the installation fails, the program crashes, runs out of time or writes an invalid file, the submission gets no score for the test set. That is why it is so important to check it first with `check_submission.py` (Section 5.1).

5.3 Uploading

Finally, compress your project folder (Section 3.3) into a zip, `team_<name>.zip`, and upload it to the Virtual Campus before the deadline. Each group uploads one zip.

6. Rules

Finally, a few rules to keep the challenge fair for everyone:

- **Any method is allowed:** classical computer vision, learning, or a mix. You can use pretrained models, public datasets and public libraries, as long as you name them in the README.
- **Your submission must run as submitted.** We do not fix code.

Última modificación: martes, 27 de octubre de 2026, 17:25

◄ Final Project. Introduction

Saltar a...

fruits_part1.zip ►

1ª UNIVERSIDAD ESPAÑOLA 40ª DEL MUNDO Rankings de Impacto de Times Higher Education 2020

universia FGUMA crue Universidades Españolas UNIVERSIDADES PÚBLICAS DE ANDALUCÍA COMUNICACIÓN RESPONSABLE

t de transparente

HR EXCELLENCE IN RESEARCH

Plan de Recuperación, Transformación y Resiliencia

Universidad de Málaga · Avda. Cervantes, 2. 29071 MÁLAGA · Tel. 952131000 · info@uma.es

Todos los derechos reservados